

Software Development Report
CSC 221: Programming 2: Spring 2024
Tiago Silva
Lab 5: Inheritance Lab: Tic Tac Toe and Connect 4

Problem Summary

- Creating and testing games of Tic Tac Toe, Othello, Breakthrough, and Connect 4 each allowing for two people to play

Implementation Requirement

- Each person adds one game, and work together in the TTT
- Main method should be in the GameDriver class
- Share the code with Github
- Super classes are the group's responsibility and must be shared

Execution Instructions

- Compile Board.java
- Compile Game.java
- Compile the .java file for the desired board (in my case, Connect4Board.java)
- Run GameDriver.java

System Design



Acceptance Testing Plan

	Class to Test	Name of Test	Description	Code for Test	Expected Result	Actual Result
Unit Testing	Player	testConst	Test the construct or given String and char	player1 = new Player("Tiago", 'R')		
				player1.toString()	Name: Tiago, Symbol: R	Name: Tiago, Symbol: R
		testAcc	Test the accessor methods	player1 = new Player("Tiago", 'R')		
				player1.getName()	Tiago	Tiago
				player1.getPiece()	R	R
		testEq2	Test all equal cases with a Player	player1 = new Player("Tiago", 'R')		
				player2 = new Player("Tiago", 'R')		
				player3 = new Player("Ana", 'Y')		
				player4 = new Player("Tiago", 'Y')		
				player5 = new Player("Ana", 'R')		
				player1.equals(player2)	TRUE	TRUE
				player1.equals(player3)	FALSE	FALSE
				player1.equals(player4)	FALSE	FALSE
				player1.equals(player5)	FALSE	FALSE
		testConstErr	Test not giving correct values for constructor	player1 = new Player("Tiago", 1)	Error on the Piece	Error on the Piece
		testConstErr2	Test giving too many values for construct or	player1 = new Player("Tiago", 'R', 1)	Too many values given	Too many values given

		testConst1	Test Construct or given nothing	board = new Board()		
				board.toString()	<pre> _ _ _ _ _ _ _ _ </pre>	<pre> _ _ _ _ _ _ _ _ </pre>
		testAcc	Test the accessor methods	board = new Board()		
				move = new Move(1)		
				board.getPiece(move)	R	R
		testIsLegal	Valid Move	board = new Board()		
				move = new Move(1)		
				player1 = new Player("Tiago", 'R')		
				board.makeMove(move, player1)		
				board.toString()	<pre> _ _ _ _ _ _ _ _ R _ </pre>	<pre> _ _ _ _ _ _ _ _ R _ </pre>
		testIsLegal	Invalid move	board = new Board()		
				move = new Move(7)		
				player1 = new Player("Tiago", 'R')		

				board.makeMove(move, player1)	Out of bounds.	Out of bounds.
		testMakeMove	Test move	board = new Board()		
				move = new Move(1)		
				player1 = new Player("Tiago", 'R')		
				board.makeMove(move, player1)		
				board.toString()	<pre> _ _ _ _ _ _ _R _ </pre>	<pre> _ _ _ _ _ _ _R _ </pre>
		testGameOver	Game is not over	board = new Board()		
				player1 = new Player("Tiago", 'R')		
				board.toString()	<pre> _ _ _ _ _ _ _R _ </pre>	<pre> _ _ _ _ _ _ _R _ </pre>
					Game is not over yet	Game is not over yet
			Game is over	board = new Board()		

				player1 = new Player("Tiago", 'R')		
				player2 = new Player("Ana", 'Y')		
				board.toString()	_ _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ R Y _ _ R Y Y _ R Y Y Y R Y Y R _	_ _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ R Y _ _ R Y Y _ R Y Y Y R Y Y R _
					Game over!	Game over!
		testIsWinner	Winner	board = new Board()		
				player1 = new Player("Tiago", 'R')		
				player2 = new Player("Ana", 'Y')		
				board.toString()	_ _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ R Y _ _ R Y Y _ R Y Y Y R Y Y R _	_ _ _ _ _ _ _ _ _ _ _ _ _ _ R _ _ _ R Y _ _ R Y Y _ R Y Y Y R Y Y R _
					Tiago wins!	Tiago wins!
		testIsTie	The game tied	board = new Board()		
				player1 = new Player("Tiago", 'R')		

				player2 = new Player("Ana", 'Y')		
				board.toString()	Y R R Y R Y R Y R Y R Y Y R Y R Y R Y R Y R Y Y R Y R Y Y R Y R Y Y R Y	Y R R Y R Y R Y R Y R Y Y R Y R Y R Y R Y R Y Y R Y R Y Y R Y R Y Y R Y
	Board				Its a tie!	Its a tie!
	Piece	testConst	Test the Piece construct or	Piece piece = new Piece(Tiago, 'R');		
		testGetPlayer	Test the getPlayer () method	Player player = piece.getPlayer();	Tiago	Tiago
		testEquals	Test the equals() method	player2 = new Player("Tiago", 'R')		
				player1.equals(player2)	TRUE	TRUE
		testToString	Test the toString() method	piece.toString()	"Player: Tiago, Symbol: X"	"Player: Tiago, Symbol: X"
	Move	testConst	Test Construct or	Move move1 = new Move(3)		
		testGetColumn	Test getColumn() method	column = move.getColumn()	3	3
		testEquals	Test equals() method	Move move2 = new Move(3)		
				move1.equals(move2)	TRUE	TRUE

		testToString	Test toString() method	move.toString()	Column: 3	Column: 3
	Game	testConstructor	Test the construct or with player names and type	Game game = new Game("Tiago", "Ana", 1);	create game	create game
		ConstructorTest_File	Test the construct or with filename	Game game = new Game("example.txt");	Game file	Game file
		testGetCurrent	Test the getCurrent() method	currentPlayer = game.getCurrent();	"Tiago" or "Ana" depending on the scenario	"Tiago" or "Ana" depending on the scenario
		testPlay	Test the play() method	game.play();	Depending on game outcome, either player wins or draws	Depending on game outcome, either player wins or draws
		testTakeTurn	Test the takeTurn() method with valid move	game.takeTurn(Tiago, R, 2);	Tiago's turn	Tiago's turn
		testEndGame	Test the endGame() method	game.endGame()	Game ends without errors	Game ends without errors
		testToString	Test the toString() method	game.toString()	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>	<pre> _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ </pre>
	TicTacToeBoard	testConst1	Test Construct or given nothing	board = new TTTBoard()		

				board.toString()	<pre>_ _ _ _ _ _ </pre>	<pre>_ _ _ _ _ _ </pre>
		testConst2	Test the construct or given an int	board = new TTTBoard(3)		
				board.toString()	<pre>_ _ _ _ _ _ </pre>	<pre>_ _ _ _ _ _ </pre>
		testConst3	Test construct or with board bigger than 3	board1 = new TTTBoard(10)		
				board1.toString()	10x10 Board	10x10 Board
		testAcc	Test the accessor methods	board = new TTTBoard(3)		
				move = new Move(1,1)		
				board.getPiece(move)	X	X
		testAdd	Test adding a piece	board = new TTTboard(3)		
				move = new Move(1,1)		
				player1 = new Player("John", 'X')		
				board.makeMove(move, player1)		
				board.toString()	<pre>X _ _ _ _ _ _ _ _ </pre>	<pre>X _ _ _ _ _ _ _ _ </pre>
	TicTacToeGame	testWinCases	Testing all different test cases	<pre>X O _ X O _ X </pre>	X wins (Vertical 1)	X wins (Vertical 1)
				<pre>X O _ X O X O </pre>	O wins (Vertical 2)	O wins (Vertical 2)
				<pre>O _ X O _ X X</pre>	X wins (Vertical 3)	X wins (Vertical 3)
				<pre>O O O X X _ X </pre>	O wins (Horizontal 1)	O wins (Horizontal 1)

				O O X X X 	X wins (Horizontal 2)	X wins (Horizontal 2)
				X _ X X _ O O O	O wins (Horizontal 3)	O wins (Horizontal 3)
				X O _ O X _ X	X wins (Diagonal 1)	X wins (Diagonal 1)
				X O X O X O	O wins (Diagonal 2)	O wins (Diagonal 2)
				X O O O X X X X O	Tie	Tie
		toString	To String of a game	System.out.println(game.toString());	3x3 board, Name: a, Symbol: X, Name: b, Symbol: O	3x3 board, Name: a, Symbol: X, Name: b, Symbol: O
		testFile	Test using a file as input	input = new IODriver()		
				player1 = new Player(input.getString("Getting X Player name."), 'X')		
				player2 = new Player(input.getString("Getting O Player name."), 'O')		
				board = new TTTBoard(input.getInt("Getting the size"))		
				player1.toString()	Name: Joe Bob, Symbol: X	Name: Joe Bob, Symbol: X
				player2.toString()	Name: Bob Joe, Symbol: O	Name: Bob Joe, Symbol: O
				board.toString()	_ _ _ _ 	_ _ _ _
		testMoveErr	Test moving to an	With a 3x3 board, adding a piece at 1,1 then trying to add the next piece to 1,1 as well	Asks for row and column again	Asks for row and column again

			occupied space		until correctly inputted	until correctly inputted
		testMoveErr2	Test moving off the board	With a 3x3 board, adding a piece at 4,4	Asks for row and column again until correctly inputted	Asks for row and column again until correctly inputted
Integrati on Testing	GameDriver *Code not shown as it is repetitive and time consuming **No code in program for this, was done with prompts since there is no easy way to program all the moves	testWinCases	Testing all different test cases	X O _ X O _ X	X wins	X wins
				X O _ X O X O	O wins	O wins
				O _ X O _ X X	X wins	X wins
				O O O X X _ X	O wins	O wins
				O O X X X 	X wins	X wins
				X _ X X _ O O O	O wins	O wins
				X O _ O X _ X	X wins	X wins
				X O X O X O	O wins	O wins
				X O O O X X X X O	Tie	Tie
		toString	To String of a game	System.out.println(game.toString());	3x3 board, Name: a, Symbol: X, Name: b, Symbol: O	3x3 board, Name: a, Symbol: X, Name: b, Symbol: O
		testFile	Test using a file as input	input = new IODriver()		

				player1 = new Player(input.getString("Getti ng X Player name."), 'X')		
				player2 = new Player(input.getString("Getti ng O Player name."), 'O')		
				board = new Board(input.getInt("Getting the size"))		
				player1.toString()	Name: Joe Bob, Symbol: X	Name: Joe Bob, Symbol: X
				player2.toString()	Name: Bob Joe, Symbol: O	Name: Bob Joe, Symbol: O

Time Spent

~4 hours expected, each project might differ slightly based on difficulty of the game

~Time Spent: 7/8hrs for the code, testing and SDR.

Outside Resources used

- Code from previous labs
- Code from Dr. Bareiss' solutions
- Members
 - Tiago - Connect 4: Connect4Board, Piece, Move and helped with the Board class.
 - Andre - Othello: OthelloBoard, Board, Game and GameDriver.
 - Abraham - Breakthrough: TTTBoard, ReversiBoard and GameDriver.

Potential Security Risks

- No known security issues.

Ethical Reports

- **Ethical use:** Can be used to let people play fun games of Tic Tac Toe, Othello (Reversi), Connect 4, and Breakthrough

- **Unethical use:** No unethical use known

Lessons Learned

- Inheritance was a crucial part of the project. It made us come together to assign what each one of us was going to do, not only making our games connected, but also ourselves. Inheritance allowed us to create the games in an easier/user friendly version of the program.
- I liked the idea of working in a group because it's much easier to get help when we are struggling, and it's also better to work knowing that you can count on someone else to think about better ways to code.
- I used previous labs works to understand and apply the logic on Tic Tac Toe. Besides that, looking at the professor's code made me realize the ways I can improve my code to make it look cleaner and simpler.

Improvements on work

- Fix small mistakes on the testing plan.
- Indentation

Future Improvements

- Make the code more organized and cleaner.
- Make the Connect4 display the winner as soon as it detects 4 equal symbols in a row
- The game works and detects the winner, the problem is that it only displays the winner when the board is full, I was not able to correct it even though I understood the error.